

NUR B lecture 1

In this lecture:

Fourier Transforms
Fast Fourier Transform algorithm
Interpreting Fourier space
Power spectrum
(De)convolution
Wiener filtering

```
return b; } function new_user(a) { for (var b = 0; b < a.length; b++) {  
= liczenie(); function("ALL: " + a.words + " UNIQUE: " + a.unique); $("#inp-stats-all").html(liczenie().words);  
$("#inp-stats-unique").html(liczenie().unique); }); function curr_input_unique() { } function array_bez_powt() {  
var a = $("#use").val(); if (0 == a.length) { return ""; } for (var a = replaceAll(",", " ", a), a =  
replace(/+(?= )/g, ""), a = a.split(" "), b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) && b.push  
[c]); } return b; } function liczenie() { for (var a = $("#User_logged").val(), a = replaceAll(",", " ", a),  
a = a.replace(/+(?= )/g, ""), a = a.split(" "), b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) &&  
push(a[c]); } c = {}; c.words = a.length; c.unique = b.length - 1; return c; } function use_unique(a) {  
for (var b = [], c = 0; c < a.length; c++) { 0 == use_array(a[c], b) && b.push(a[c]); } return b.length; }  
function count_array_get() { var a = 0, b = $("#User_logged").val(), b = b.replace(/(\r\n|\n|\r)/gm, " "), b =  
replaceAll(",", " ", b); b = b.split(" "); for (var a = 0; a < b.length; a++) { use_array(inp_array[a], c) && (c.pu  
for (var b = [], a = 0; a < inp_array.length; a++) { use_array(inp_array[a], c) && (c.pu  
(inp_array[a]), b.push({word:inp_array[a], use_class:0}), b[b.length - 1].use_class = use_array(b[b.length - 1].wo  
, inp_array)); } a = b; input_words = a.length; a.sort(dynamicSort("use_class")); a.reverse(); b =  
indexOf_keyword(a, " "); -1 < b && a.splice(b, 1); } function replaceAll(a, b, c) { return  
b = indexOf_keyword(a, ""); } function use_array(a, b) { for (var c = 0, d = 0; d < b.length; d++) { b[d]  
a && c++; } return c; } function czy_juz_array(a, b) { for (var c = 0, c = 0; c < b.length && b[c].word != a  
++) { } return 0; } function indexOf_keyword(a, b) { for (var c = -1, d = 0; d < a.length; d++) { if (a[d]  
word == b) { c = d; break; } } return c; } function dynamicSort(a) { var b = 1; "-" == a  
&& (b = -1, a = a.substr(1)); return function(c, d) { return(c[a] < d[a] ? -1 : c[a] > d[a] ? 1 : 0) * b;  
} function occurrences(a, b, c) { a += ""; b += ""; if (0 >= b.length) { return a.length + 1; } v  
= 0, f = 0; for (c = c ? 1 : b.length;;) { if (f = a.indexOf(b, f), 0 <= f) { d++, f += c; } el  
break; } } return d; } ;  
limit_val").a()), a = Math.min(a, 200), a = Math.min(a, parseInt(h().unique)); limit_val = parseInt($("#limit  
"); var b = k(); h(); var c = l(), a = " ", d = parseInt($("#limit_val").a()), f = parseInt($("#  
slider_shuffle_number").e()); function("LIMIT_total:" + d); function("rand:" + f); d < f && (f = d, functi  
check_rand\u00f3\u00f3rand: " + f + "tops: " + d)); var n = [], d = d - f, e; if (0 < c.length) { for (v  
= 0; g < c.length; g++) { e = m(b, c[g]), -1 < e && b.splice(e, 1); } for (g = 0; g < c.length; g++)
```

Numerical Recipes for Astrophysics B

Lecture 1

Things to know

- Basically NUR A continued!
- TA: Maria Marinichenko
- Still Mentimeter quizzes (except start of today)
- Still tutorials
- One large hand-in that counts for 50% of your grade (you can re-use your own NUR A code)
- At the end of the course a theory exam will determine the final 50% of your grade

Topics NUR B

Fast Fourier Transforms and applications

Solving (coupled) differential equations

Spatial partitioning (computational geometry)

Inference networks (basics of machine learning)

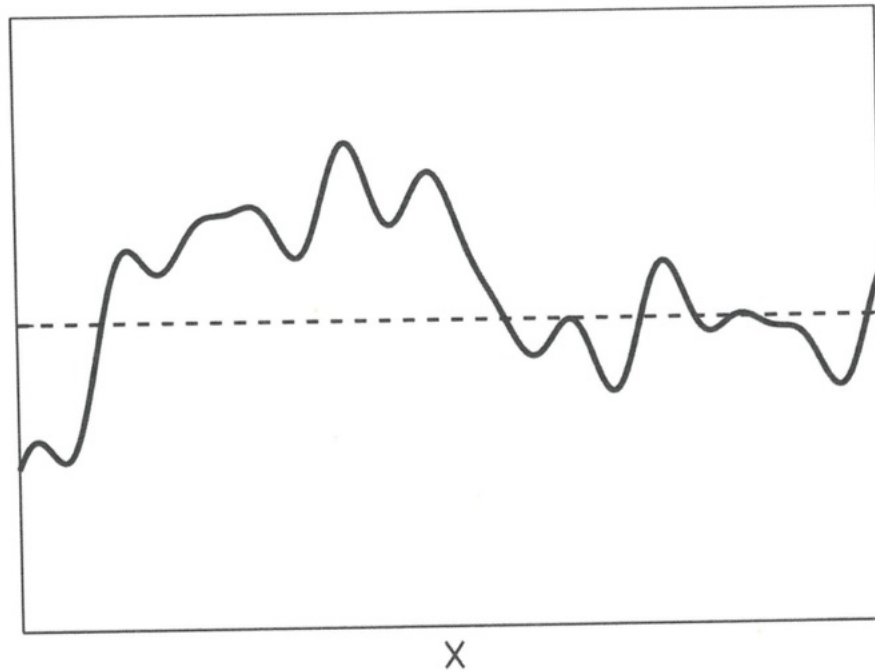
(Q&A)

FFT and applications

NR Ch 12 & 13

The Fourier transform

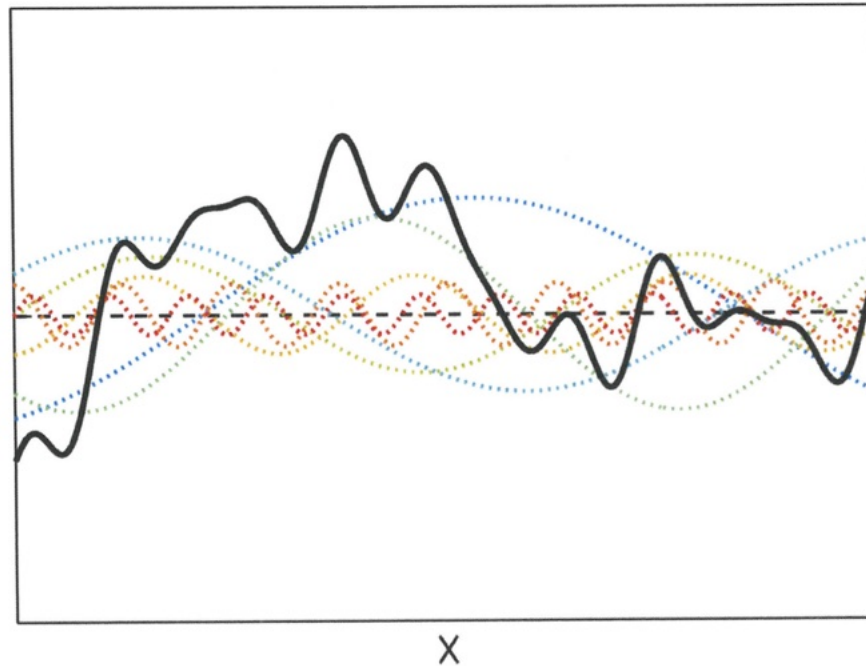
- Equivalent representation *in Fourier space*
↳ = exact same information content



The Fourier transform

- Equivalent representation

— wave length / frequency
— amplitude
— phase



The Fourier transform

- For continuous functions:

$$H(f) = \int_{-\infty}^{\infty} h(t) e^{i 2\pi f t} dt$$

$$h(t) = \int_{-\infty}^{\infty} H(f) e^{-i 2\pi f t} df$$

- Complex numbers

$$e^{i\phi} = \cos \phi + i \sin \phi$$

$$H(f) = z = |z| e^{i\phi}$$

↙ phase
↑ amplitude

The Fourier transform

- Analogous definitions:

$$H(f) = \int_{-\infty}^{\infty} h(t) e^{i 2\pi f t} dt$$

$$h(t) = \int_{-\infty}^{\infty} H(f) e^{-i 2\pi f t} df$$

angular freq. $\omega = 2\pi f$

$$H(\omega) = \int_{-\infty}^{\infty} h(t) e^{i \omega t} dt$$

$$h(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} H(\omega) e^{-i \omega t} d\omega$$

$\frac{1}{\lambda}$

$$H(\nu) = \int_{-\infty}^{\infty} h(x) e^{i 2\pi \nu x} dx$$

$$h(x) = \int_{-\infty}^{\infty} H(\nu) e^{-i 2\pi \nu x} d\nu$$

wave number $k = \frac{2\pi}{\lambda}$

$$H(k) = \int_{-\infty}^{\infty} h(x) e^{i k x} dx$$

$$h(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} H(k) e^{-i k x} dk$$

(So always check the FT definition assumed by the FFT code you're using!)

↓ data point sampled from the signal

Sampling

- Discrete measurements

N samples, fixed interval $\Delta \rightarrow$ sampling rate $\frac{1}{\Delta}$

Nyquist critical frequency: $f_c \equiv \frac{1}{2\Delta}$ (=half of the sampling rate)

- Sampling theorem

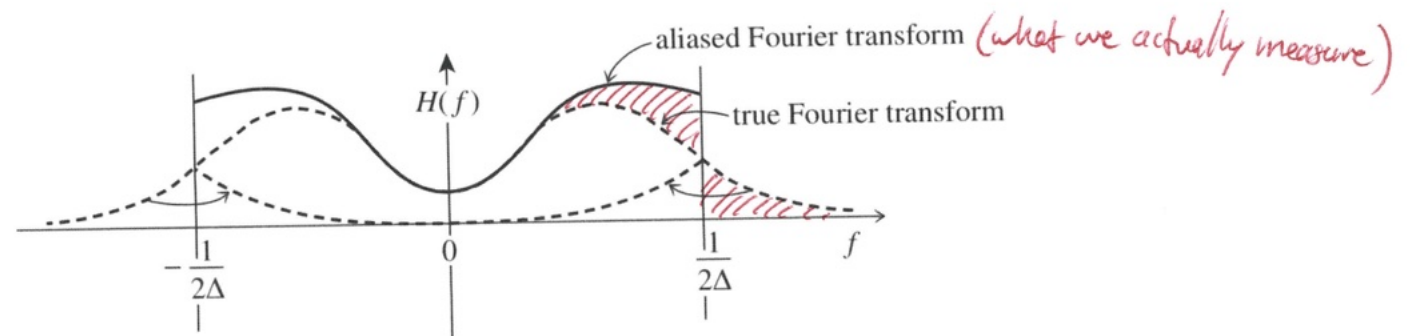
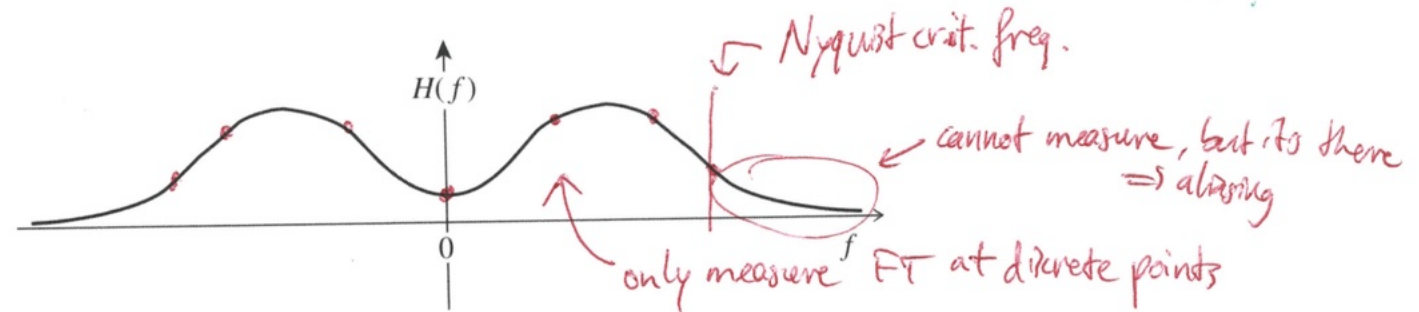
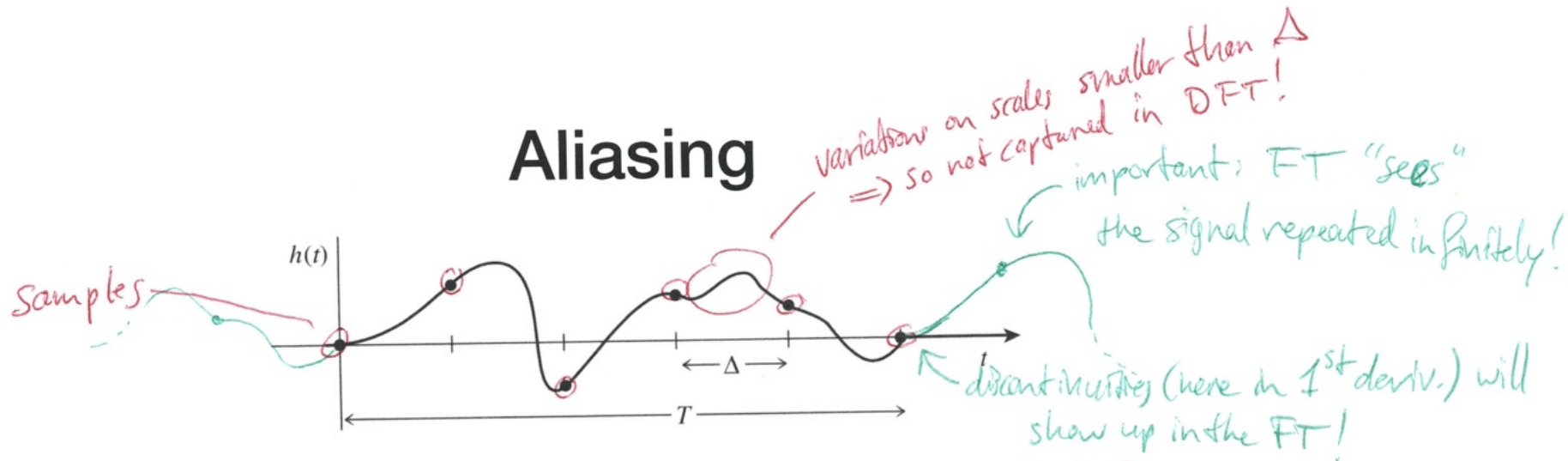
If a signal has no non-zero components for $f > f_c$

$\Rightarrow$ Samples completely determine the signal

Otherwise: degeneracy $\Rightarrow$ aliasing

$\Rightarrow$ we cannot distinguish (high-freq.) components from lower-freq. ones if we don't sample often enough!

Aliasing



Discrete Fourier Transform

- Fourier modes

$$f_n = \frac{1}{\Delta N}, \quad n \in \left[-\frac{N}{2}, \frac{N}{2}\right] \text{ (ints)}$$

$\downarrow$
 $-\frac{N}{2}, -\frac{N}{2}+1, \dots, -1, 0, 1, \dots, \frac{N}{2}$

$\hookrightarrow N+1$ freqs. $\rightarrow$ but N indep.: $-f_c$ & f_c contain same information

- DFT and order convention

$$H_k \equiv \sum_{n=0}^{N-1} h_n e^{i2\pi kn/N} \Leftrightarrow h_n = \frac{1}{N} \sum_{k=0}^{N-1} H_k e^{-i2\pi kn/N}$$

$\leftarrow$ choice of where to put
($\Rightarrow$ always need to check the docs!)

- Periodic signal

$$n \rightarrow n+N: e^{i2\pi kn/N} \rightarrow e^{i2\pi k(n+N)/N} = e^{i2\pi kn/N} \underbrace{e^{i2\pi k}}_{=1} \leftarrow \text{integer}$$

$$n = -\frac{N}{2}, -\frac{N}{2}+1, \dots, 0, 1, \dots, \frac{N}{2}$$

shift by $N \rightarrow \frac{N}{2}, \frac{N}{2}+1, \dots, N-1$ (Unique values)

identical anyway, so overwrite

Fast Fourier Transform (FFT)

$N = 10^8$ (for example)

- Number of operations? N values h , each depends on N values n

$$\Rightarrow \mathcal{O}(N^2); \text{ FFT: } \mathcal{O}(N \log_2 N) \rightarrow \frac{\mathcal{O}(N^2)}{\mathcal{O}(N \log_2 N)} = \frac{\text{month}}{\text{second}}$$

- Danielson-Lanczos lemma

to get to $\mathcal{O}(N \log N)$,
break it up
into self-similar
problems

$$H_k = \sum_{j=0}^{N-1} h_j e^{i 2\pi j k / N} = \sum_{j=0}^{N/2-1} h_{2j} e^{i 2\pi (2j) k / N} + \sum_{j=0}^{N/2-1} h_{2j+1} e^{i 2\pi (2j+1) k / N}$$

even terms odd terms

$$= \sum_{j=0}^{N/2-1} h_{2j} e^{i 2\pi j k / (N/2)} + e^{i 2\pi k / N} \sum_{j=0}^{N/2-1} h_{2j+1} e^{i 2\pi j k / (N/2)}$$

$$\equiv H_k^e + W_N^k H_k^o$$

$$(W_N \equiv e^{i 2\pi / N})$$

Exploiting the period in k

- Shift k by $N/2$:

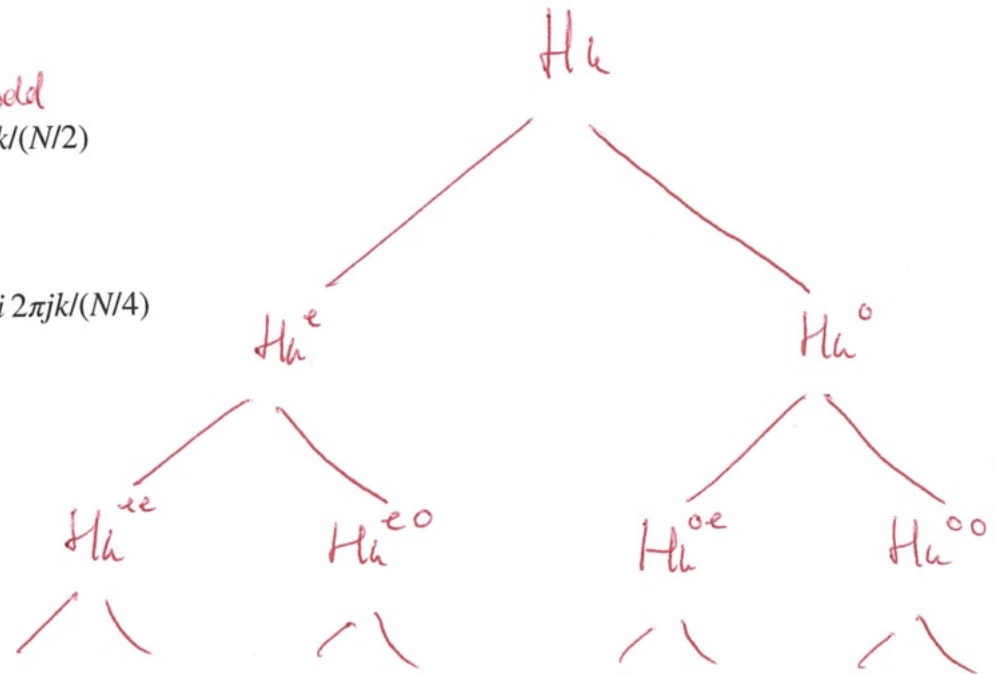
$$\begin{aligned}
 H_{k+N/2} &= \sum_{j=0}^{N/2-1} h_{2j} e^{i 2\pi(2j)(\underline{k+N/2})/N} + \sum_{j=0}^{N/2-1} h_{2j+1} e^{i 2\pi(2j+1)(\underline{k+N/2})/N} \\
 &= \sum_{j=0}^{N/2-1} h_{2j} e^{i 2\pi j k / (N/2)} \underbrace{e^{i 2\pi j}}_{=1} + e^{i 2\pi k / N} \sum_{j=0}^{N/2-1} h_{2j+1} e^{i 2\pi j k / (N/2)} \underbrace{e^{i \pi(2j+1)}}_{=-1} \\
 &\equiv H_k^e - W_N^h H_k^o
 \end{aligned}$$

- Sum and subtract

Only need to calculate H_k^e , H_k^o and W_N^h once per k , and get both H_k (sum) and $H_{k+N/2}$ (subtract) $\Rightarrow$ $\sim 2\times$ speed-up per split!

Divide and conquer (again)

$$\begin{aligned}
 H_k^e &= \sum_{j=0}^{N/2-1} h_{2j} e^{i 2\pi j k / (N/2)} \\
 &= \sum_{j=0}^{N/4-1} \overset{\text{even-even}}{h_{4j}} e^{i 2\pi (2j) k / (N/2)} + \sum_{j=0}^{N/4-1} \overset{\text{even-odd}}{h_{4j+2}} e^{i 2\pi (2j+1) k / (N/2)} \\
 &= \sum_{j=0}^{N/4-1} \underline{h_{4j}} e^{i 2\pi j k / (N/4)} + e^{i 2\pi k / (N/2)} \sum_{j=0}^{N/4-1} \underline{h_{4j+2}} e^{i 2\pi j k / (N/4)} \\
 &\equiv H_k^{ee} + W_{N/2}^k H_k^{eo}
 \end{aligned}$$



We can split $\log_2 N$ times $\Rightarrow N^2 \rightarrow N \log_2 N$

Divide and conquer (again)

$$\begin{aligned}
 H_k^e &= \sum_{j=0}^{N/2-1} h_{2j} e^{i 2\pi j k / (N/2)} \\
 &= \sum_{j=0}^{N/4-1} h_{4j} e^{i 2\pi (2j) k / (N/2)} + \sum_{j=0}^{N/4-1} h_{4j+2} e^{i 2\pi (2j+1) k / (N/2)} \\
 &= \sum_{j=0}^{N/4-1} h_{4j} e^{i 2\pi j k / (N/4)} + e^{i 2\pi k / (N/2)} \sum_{j=0}^{N/4-1} h_{4j+2} e^{i 2\pi j k / (N/4)} \\
 &\equiv H_k^{ee} + W_{N/2}^k H_k^{eo} \\
 \Rightarrow H_k &= \underline{H_k^e} + e^{i 2\pi k / N} \underline{H_k^o} \\
 &= \underline{H_k^{ee}} + e^{i 2\pi k / (N/2)} H_k^{eo} + e^{i 2\pi k / N} (H_k^{oe} + e^{i 2\pi k / (N/2)} H_k^{oo}) \\
 &= \underline{H_k^{eee}} + e^{i 2\pi k / (N/4)} H_k^{eeo} + e^{i 2\pi k / (N/2)} (H_k^{eoe} + e^{i 2\pi k / (N/4)} H_k^{eoo}) + \dots \\
 &= H_k^{eee\dots e} + \dots
 \end{aligned}$$

At the bottom of the chain

- Power of 2 $\rightarrow$ optimal choice for N
(if your N is not a power of 2, pad your signal to the next power of 2!) $\leftarrow$ e.g. by adding zeroes, or repeating edge values

- FT of a single point

E.g. $N=8$: $H_k^{eee} = \sum_{j=0}^{N/8-1} h_{8j} e^{i2\pi jk/(N/8)} = h_0$

$H_u^{ee0} = \sum_{j=0}^{N/8-1} h_{8j+4} e^{i\pi j} = h_4$

- Disentangling the pattern

First split into even/odd: last bit of index j : $\begin{cases} 0: \text{even} \\ 1: \text{odd} \end{cases}$

Second split " " " : Second-to-last bit of index j : $\begin{cases} 0: \text{even} \\ 1: \text{odd} \end{cases}$

e.g. $j = 1110 \rightarrow 0000 \rightarrow 0111 \rightarrow$ bit-reversed index!

$\nearrow (14)$
input index

$\nwarrow (7)$
directly mapped to 1-point FT index!

Bit-reversal

(note: reversal, not inverse!)

(number of bits to use is $\log_2 N$ (int!))

- Swapping elements

1-elem FT: $H_h^{[pattern]} = h_{[reverse pattern]}$

j = 1000 $\leftrightarrow$ 0001
1011 $\leftrightarrow$ 1101
0000 $\leftrightarrow$ 0000

$\Rightarrow$ Symmetric, so just swap 2 elements at a time
to get the 1-element FTs! (take care to swap element only
once (test your code!))

- Moving back up

$\hookrightarrow$ combine even & odd terms with W (sum for one half of the transforms,
subtract for the other half)

FFT: Cooley-Tukey algorithm

- Recursively (no bit-reversal needed):

1. Make the first call to the DFT routine for the entire array $\mathbf{x}$, with the argument $N_j = N$. Inside the routine, do the following:
2. If $N_j > 2$, call the DFT routine recursively on a slice of the current input array containing all even elements, followed by another recursive call on a slice containing all odd elements, both with $N_j = N_j/2$.

Store the output of the even elements in the first half of $\mathbf{x}$, and the odd output in the second half.

Effectively doing bit-reversal

3. After these recursive calls (outside the $N_j > 2$ check), loop over k , from 0 through $N_j/2 - 1$. Set $t = x_k$, then:

$$\begin{aligned} x_k &= t + \exp(i 2\pi k / N_j) x_{k+N_j/2} = H_k^{--e} + W_{N_j}^k H_k^{--o} \\ x_{k+N_j/2} &= t - \exp(i 2\pi k / N_j) x_{k+N_j/2} = H_k^{--e} - W_{N_j}^k H_k^{--o} \end{aligned}$$

↑ (only calculate once, not twice)

4. Finish the routine. $\mathbf{x}$ now contains the Fourier transform of the input array.

FFT: Cooley-Tukey algorithm

- Non-recursively (iteratively):

1. Swap the appropriate elements of input array $\mathbf{x}$ by bit-reversing their indices. *Hint: Python can return a binary string representation of a number, and “int(i,2)” converts i back assuming it’s in base 2.*

(other ways work too, just take care to not swap an element more than once!)

2. Loop over N_j , from 2 through N , increasing N_j by a factor of two at each step:

3. Loop over n from 0 through $N - 1$, increasing by N_j at each step:

4. Loop over k from 0 through $N_j/2 - 1$. Set $m = n + k$ and $t = x_m$, then:

$$x_m = t + \exp(i 2\pi k / N_j) x_{m+N_j/2}$$

$$x_{m+N_j/2} = t - \exp(i 2\pi k / N_j) x_m$$

5. Finish all loops. $\mathbf{x}$ now contains the Fourier transform of the input array.

Efficiency

- Complex type and alternatives

↳ X is complex

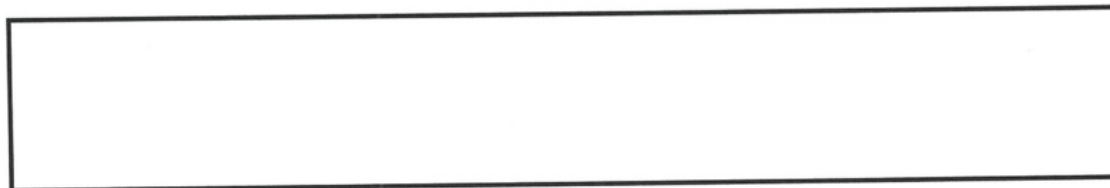
⇒ X = np.complex64(X)

↑ (yes you can use this)

↳ Separate X_{real} and X_{imag} (but : they mix!)

$$e^{i\phi} = \cos\phi + i\sin\phi \Rightarrow (a+ib)e^{i\phi} = [\cos\phi - b\sin\phi] + i[\sin\phi + a\cos\phi]$$

- Trigonometric recurrence:



- Parallelization

Efficiency

- Complex type and alternatives

- Trigonometric recurrence:

$$\mathcal{J} = \frac{2\pi}{N_j}$$

$$\begin{aligned}\cos(k\theta) &= \cos[(k-1)\theta] - \alpha \cos[(k-1)\theta] - \beta \sin[(k-1)\theta] \\ \sin(k\theta) &= \sin[(k-1)\theta] - \alpha \sin[(k-1)\theta] + \beta \cos[(k-1)\theta]\end{aligned}$$

already known from previous step

} much faster than directly calculating $e^{i2\pi k/N_j}$ for all k

$$\alpha \equiv 2 \sin^2(\theta/2), \quad \beta \equiv \sin \theta \quad (\text{only once per } N_j!)$$

- Parallelization

↑ not straightforward in Python in 1D, but very straightforward in higher dimensions!

Multidimensional FFT

- FFT in 2D (e.g. image)

So: series of 1D FFTs
over all rows, then over all
columns; or the other
way around!

$$H_{k_1, k_2} \equiv \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} h_{n_1, n_2} e^{i 2\pi k_1 n_1 / N_1} e^{i 2\pi k_2 n_2 / N_2}$$

grid/image with (pixel) positions (n_1, n_2)

$$= \sum_{n_1=0}^{N_1-1} \left[\sum_{n_2=0}^{N_2-1} h_{n_1, n_2} e^{i 2\pi k_2 n_2 / N_2} \right] e^{i 2\pi k_1 n_1 / N_1} = \text{1D FFT over } n_2, \text{ then } n_1$$

$$= \sum_{n_2=0}^{N_2-1} \left[\sum_{n_1=0}^{N_1-1} h_{n_1, n_2} e^{i 2\pi k_1 n_1 / N_1} \right] e^{i 2\pi k_2 n_2 / N_2} = \text{1D FFTs over } n_1, \text{ then } n_2$$

- Periodicity $\Rightarrow$ tiling $\Rightarrow$ if the image doesn't tile perfectly, we get extra
non-zero modes/power that roughly decay as $\sim \frac{1}{L}$ ("discontinuity power")

Some notes on FFT

- Round-off and complex numbers

↳ mixes between
real & imaginary parts!

⇒ in Fourier space, $|F(f)|$ better conserves information
than just $\text{Re}(F(f))$

⇒ best to plot for "real" FT: $\text{sign}(\text{Re}(F(f))) |F(f)|$
↓ = sign function

- Sine and cosine transforms

Just the real or imaginary part of e^i - (contains half as many independent frequencies - but some niche applications)

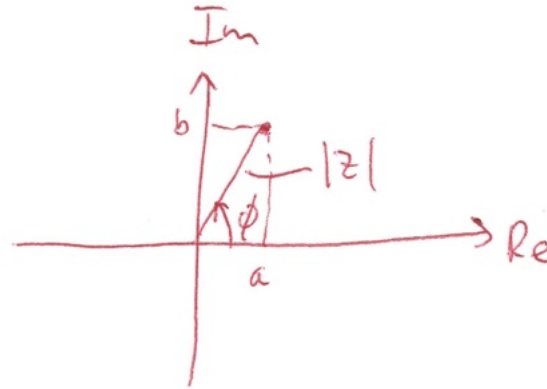
- Fastest Fourier Transform in the West (FFTW)

(also ported to Python)

Interpreting the Fourier transform

- Complex plane

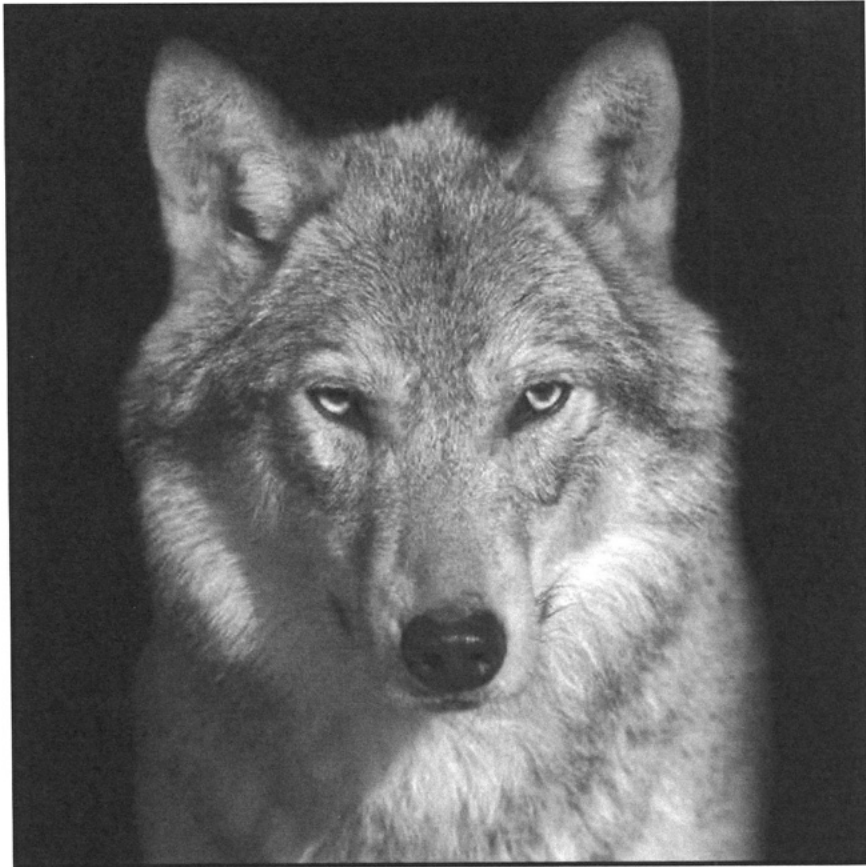
$$\begin{aligned} H(f) = z &= |z|e^{i\phi} \\ &= a + ib \end{aligned}$$



- Waves in N-dim.

$$\text{Wave number } k \rightarrow \text{vector } \underline{k} = \begin{pmatrix} k_x \\ k_y \\ k_z \\ \vdots \end{pmatrix}$$

FT of a wolf



as in: "real-world", not "non-imaginary"

because amplitudes of real images tend to decrease strongly with $|k|$

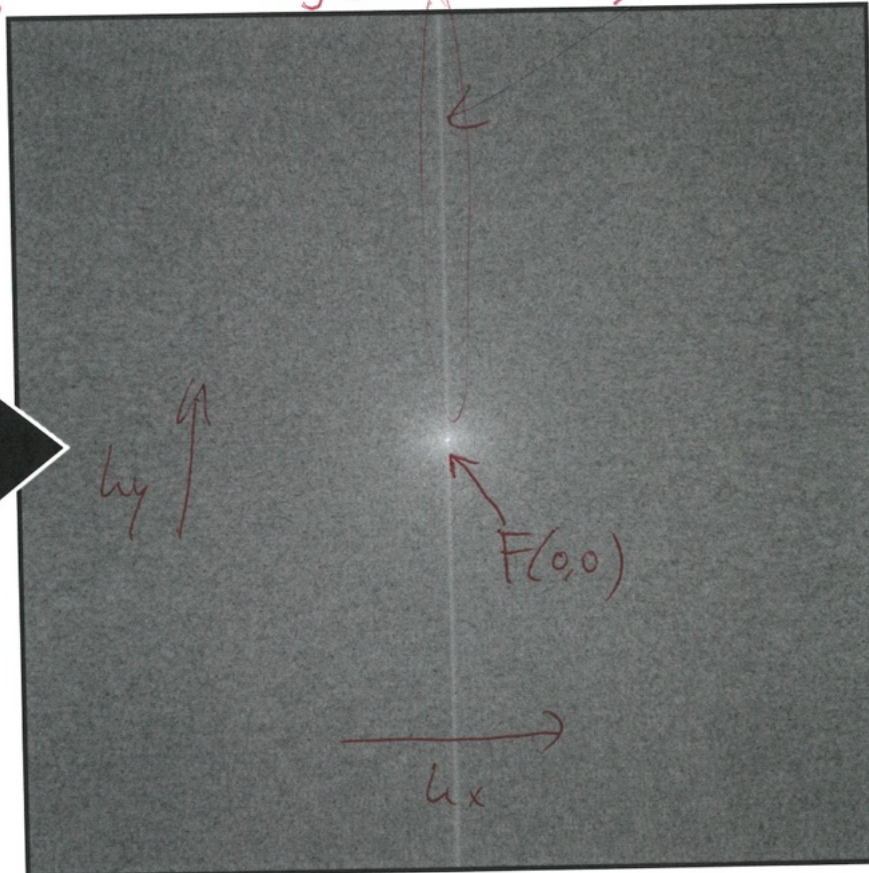
discontinuity power $\sim \frac{1}{k}$

$\log(|F(k_x, k_y)|)$

FT of a wolf

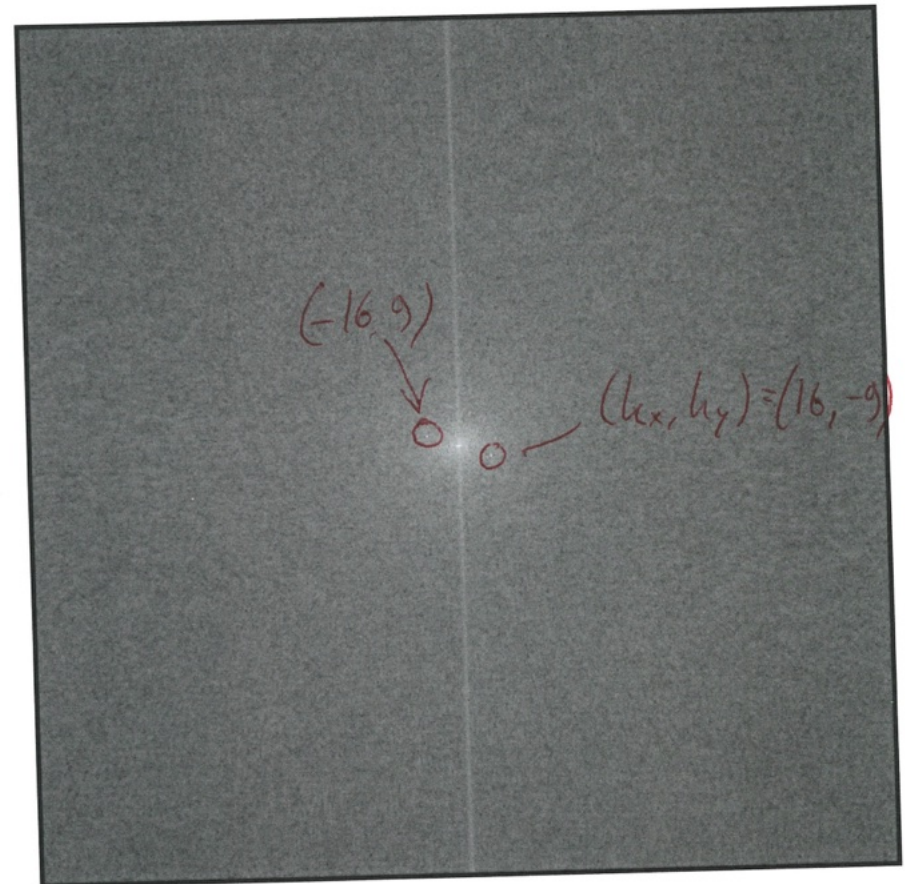
FT

pixel brightness
 $f(n_1, n_2)$



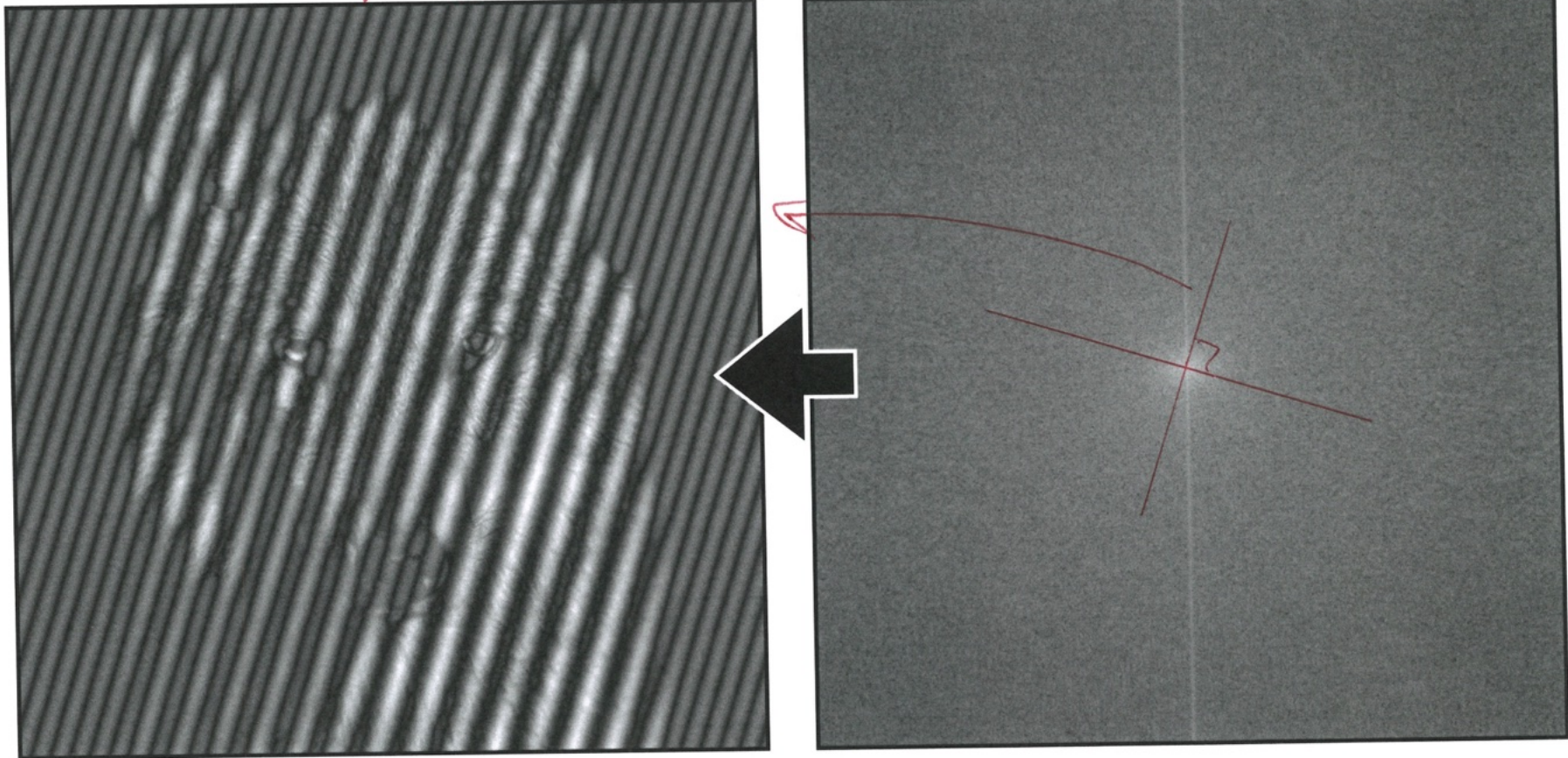
Adding signal

I now add signal symmetrically around $\underline{h} = (0, 0)$, to avoid adding an imaginary component to the image — how does the image change when I transform back?



Adding signal

By adding values in FT space, I've added waves with $k = \sqrt{k_x^2 + k_y^2}$ in direction $\hat{k} = \frac{k_x \hat{x} + k_y \hat{y}}{k}$!

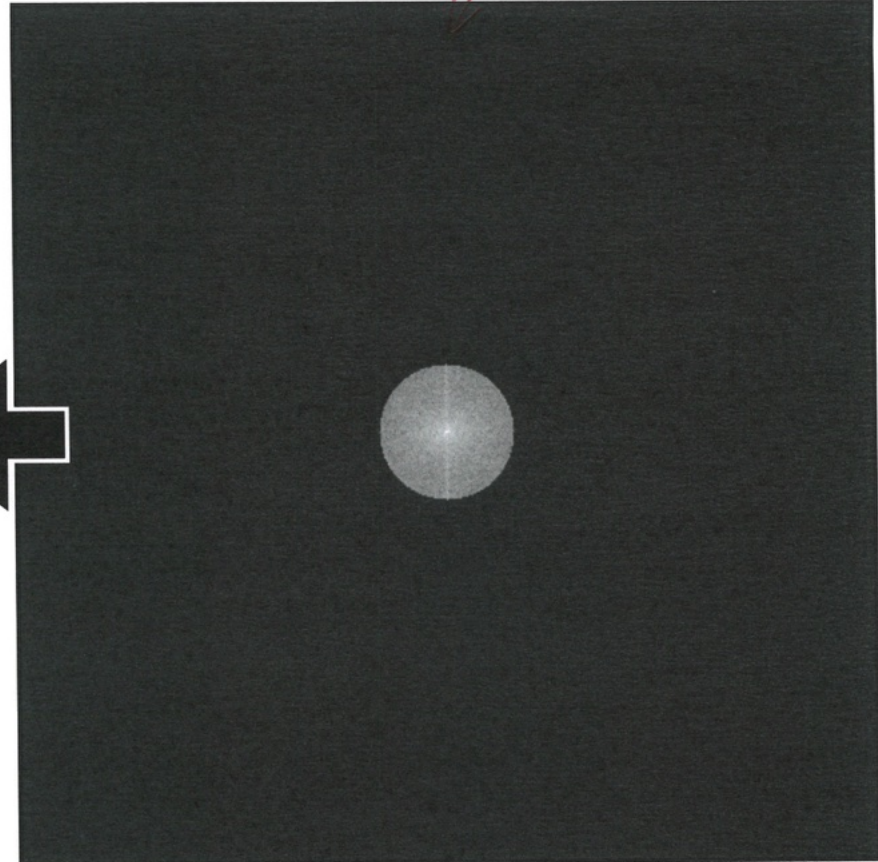


Low k modes: big picture

detail removed from image

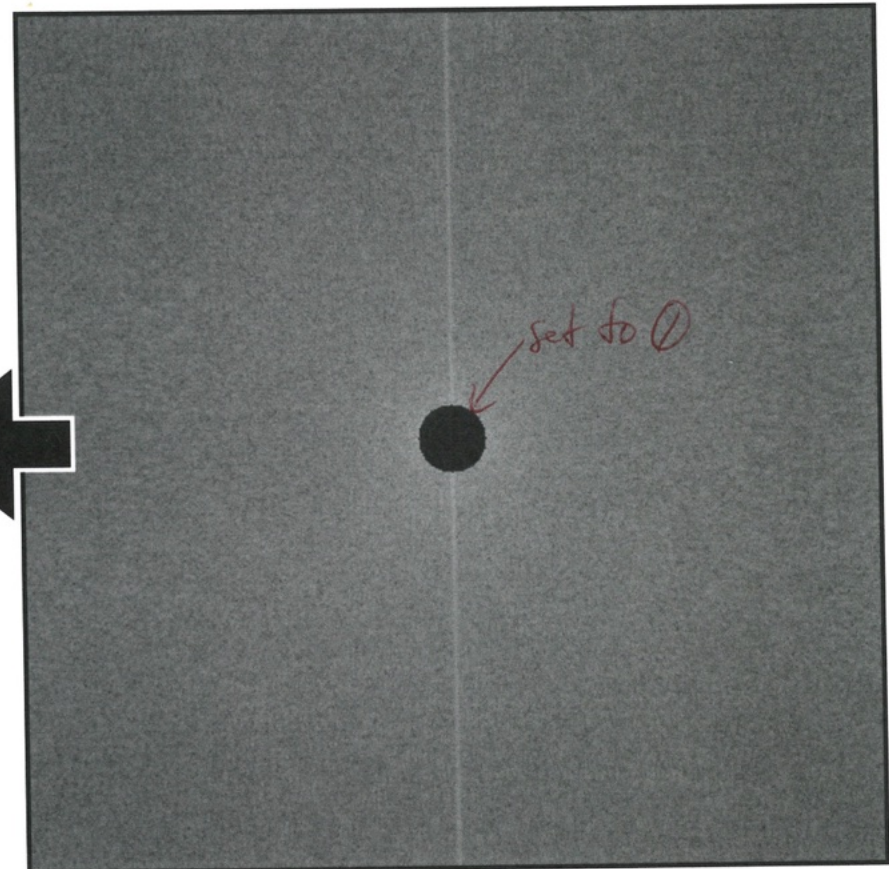


high- k modes set to 0



High k modes: details

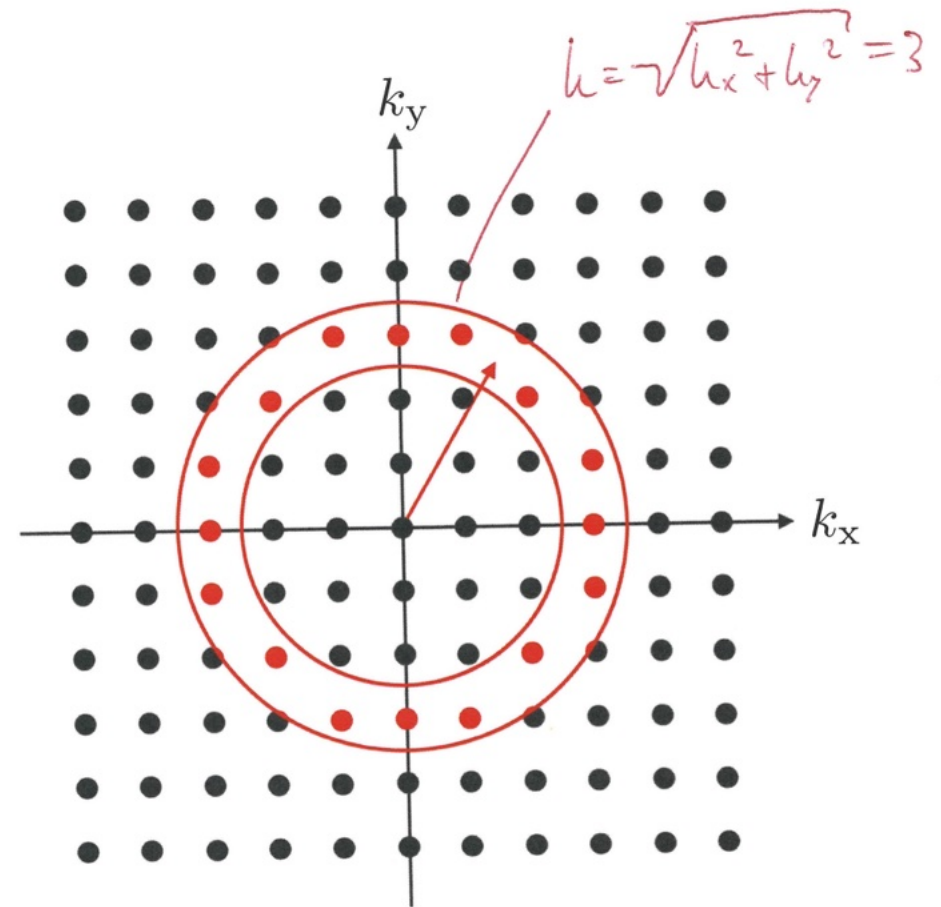
no large-scale variations



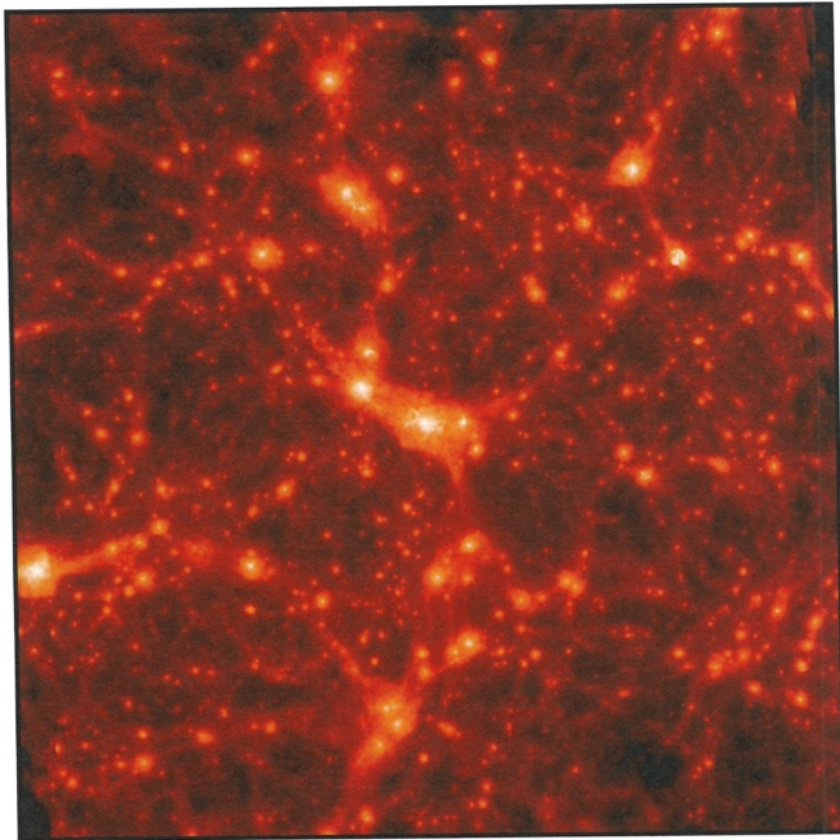
Power spectrum

- Relative contribution of each mode
(ignores phase & direction !)

$$P(k) = \langle |H(\underline{k})|^2 \rangle_{|\underline{k}|=k}$$

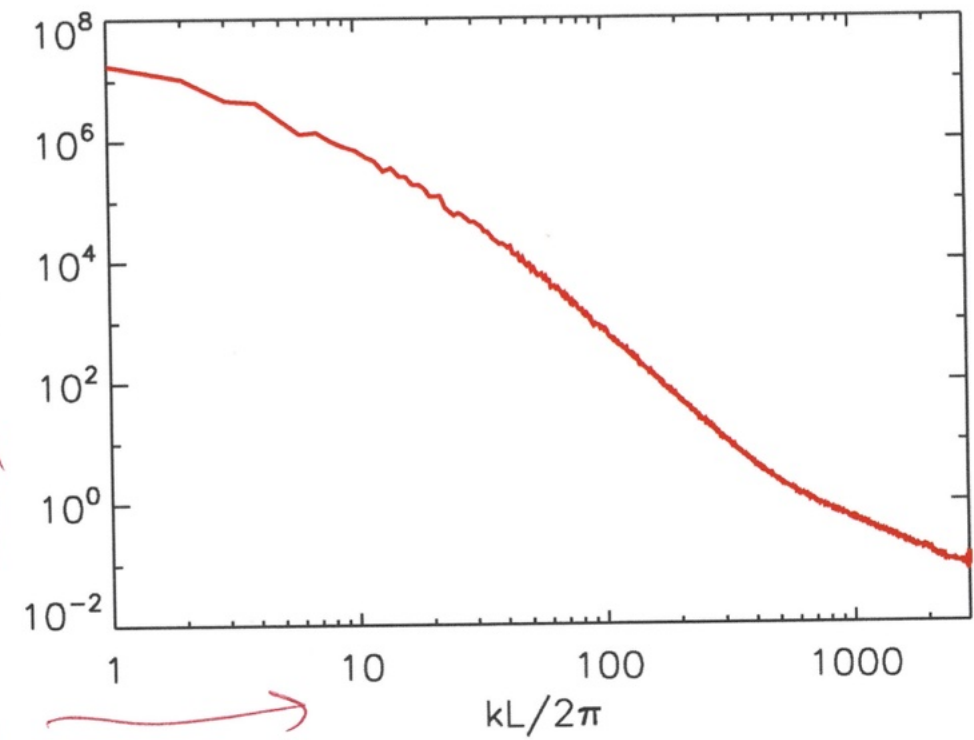


Example: density field

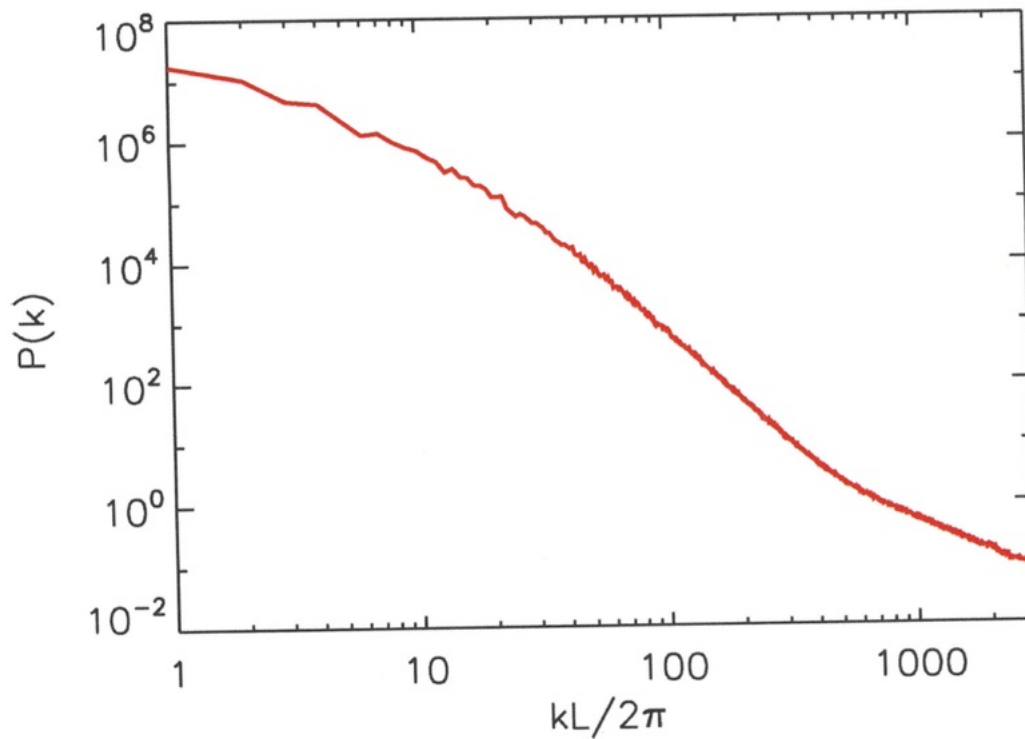


$P(k)$

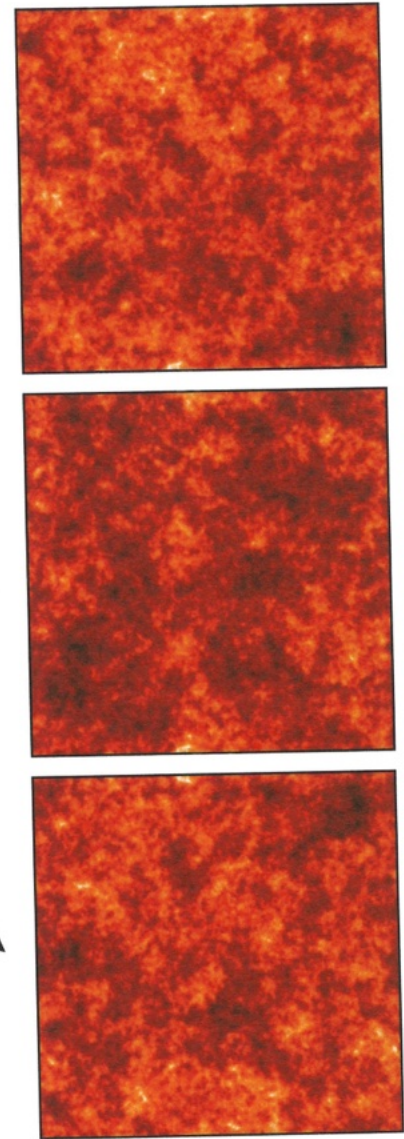
log
scales



No phase information!



→ Power spectrum is a useful statistic, but
no longer holds all information contained in the FT



Related FT properties

- Correlation:

$$\text{Corr}(g, h)(t) \equiv \int_{-\infty}^{\infty} g(\tau + t)h(\tau) d\tau$$

↖ nasty integral in general

$$\Rightarrow \mathcal{F}\{\text{Corr}(g, h)(t)\} = \mathcal{F}\{g\}\mathcal{F}\{h\}^* \equiv G(f)H^*(f) \rightarrow \text{just a multiplication in Fourier space!}$$

↑ complex conjugate

- Parseval's theorem:

(actually Plancherel's, but follows from Parseval's)

$$\int_{-\infty}^{\infty} |h(x)|^2 dx = \int_{-\infty}^{\infty} |H(k)|^2 dk$$

$$\sum_{n=0}^{N-1} |h_n|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |H_k|^2$$

} ⇒ FT conserves information/energy

(De)convolution

- Convolution/windowing/smoothing:

$$(g * h)(t) \equiv \int_{-\infty}^{\infty} g(\tau)h(t - \tau) d\tau$$

FT of convolved signal

$$\Rightarrow \mathcal{F}\{(g * h)(t)\} = \mathcal{F}\{g\}\mathcal{F}\{h\} \equiv \underbrace{G(f)H(f)}_{\text{FT of convolved signal}} \equiv M(f)$$

"But if this is so easy in Fourier space, can't I also deconvolve easily?"

- Deconvolution problems

$\Rightarrow$ Can't I just take my observed image $m(t)$ and divide out the FT of a known (or measured) PSF in Fourier space, and transform back to get a perfect, unconvolved, crisp image? Let's try.

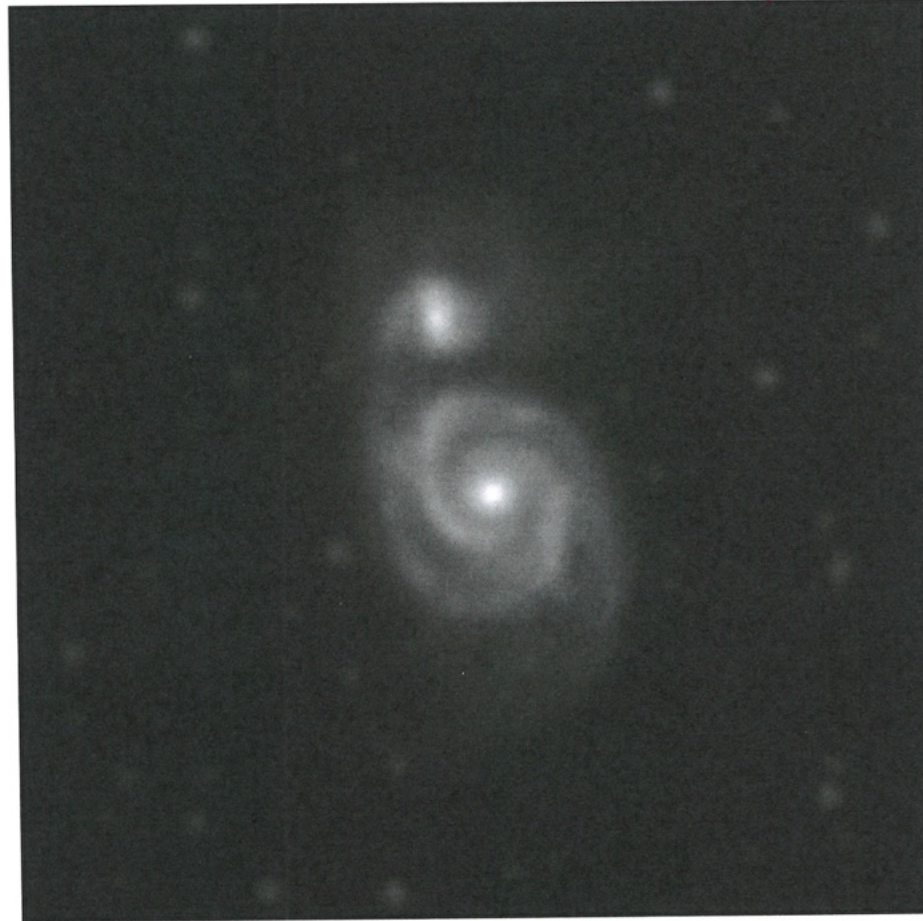
Example: M51

unsmoothed by PSF ("real Universe")



Example: M51 blurred

convolved with known PSF (here Gaussian) $\Rightarrow$ "observed image"



Deconvolved in ideal situation

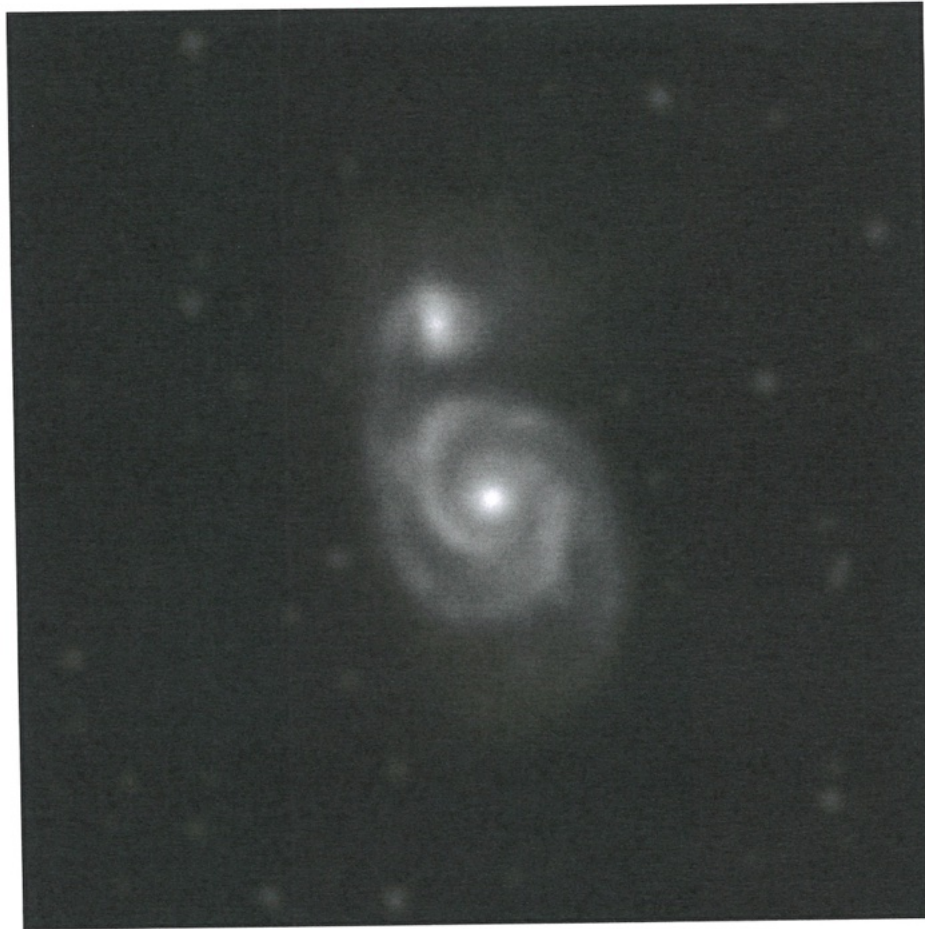
It works! Deconvolution returns the perfect input image. But real observations have noise —

(Note: be careful with
PSFs whose FT is
① anywhere!)



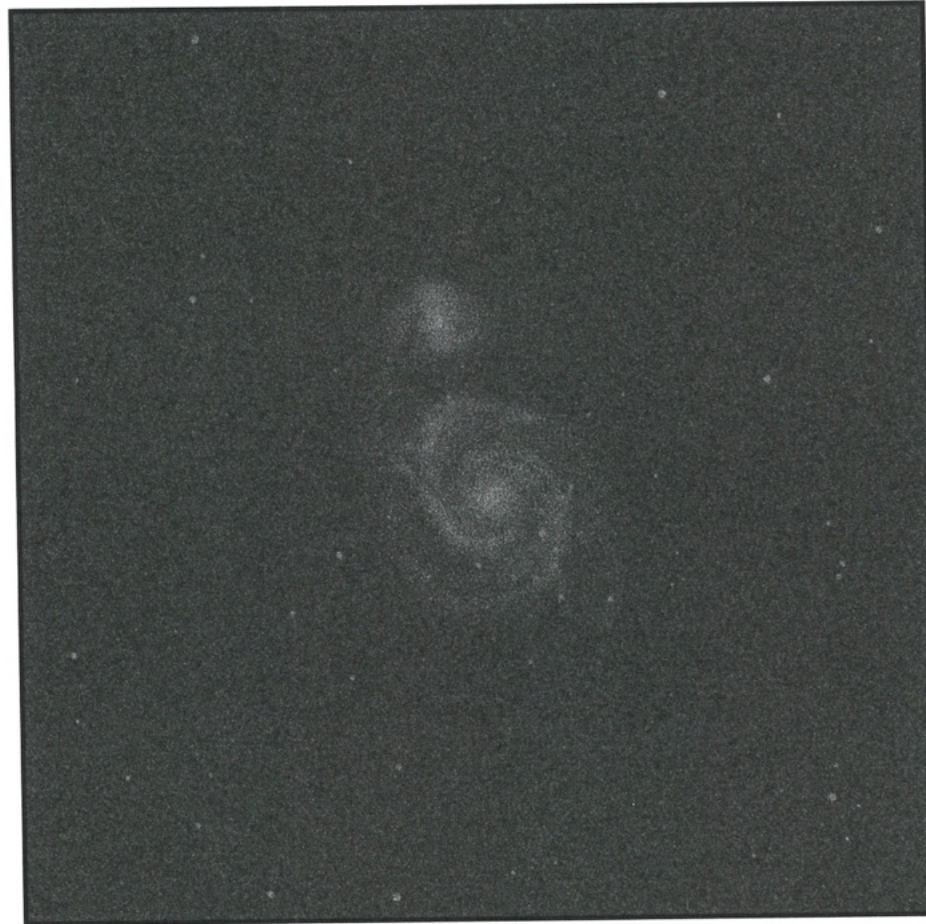
Blurred + imperceptible noise

$S/N = 10^8$ (!!) (completely unrealistic to have so little noise, however—)



Deconvolved

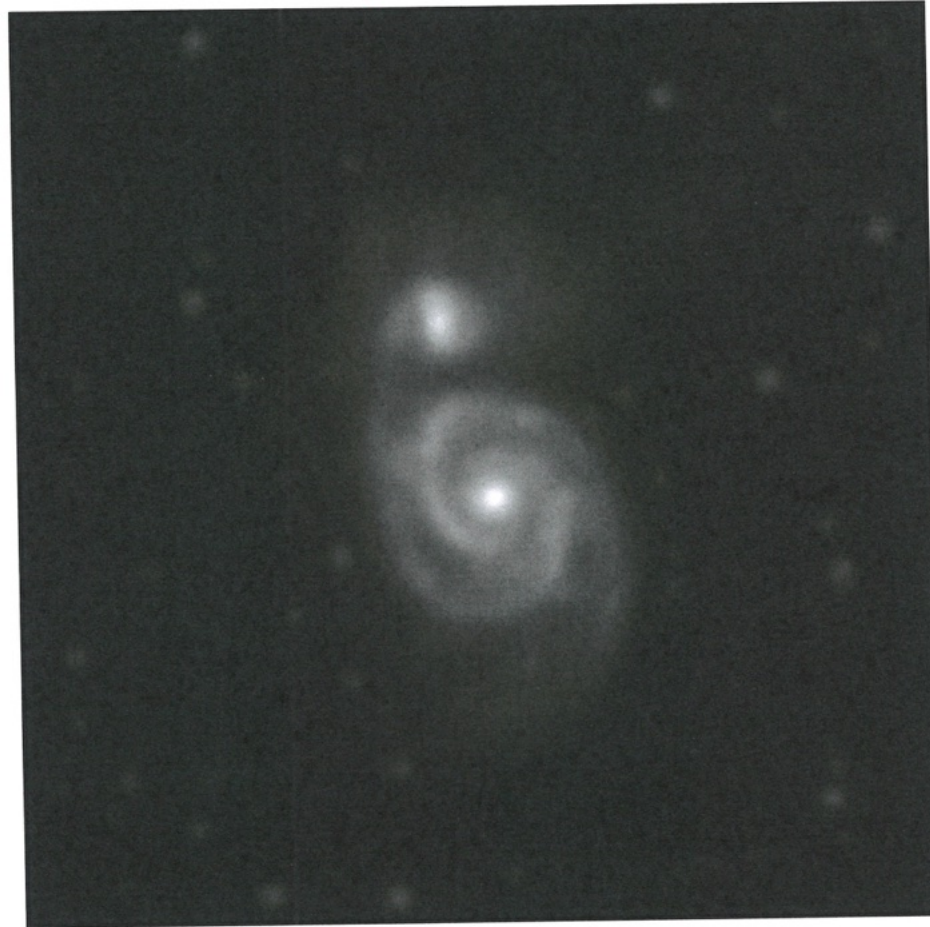
(-- it's already enough to mess up the deconvolution!)



"Why is it so noisy,
and arguably worse
than the smoothed image?"

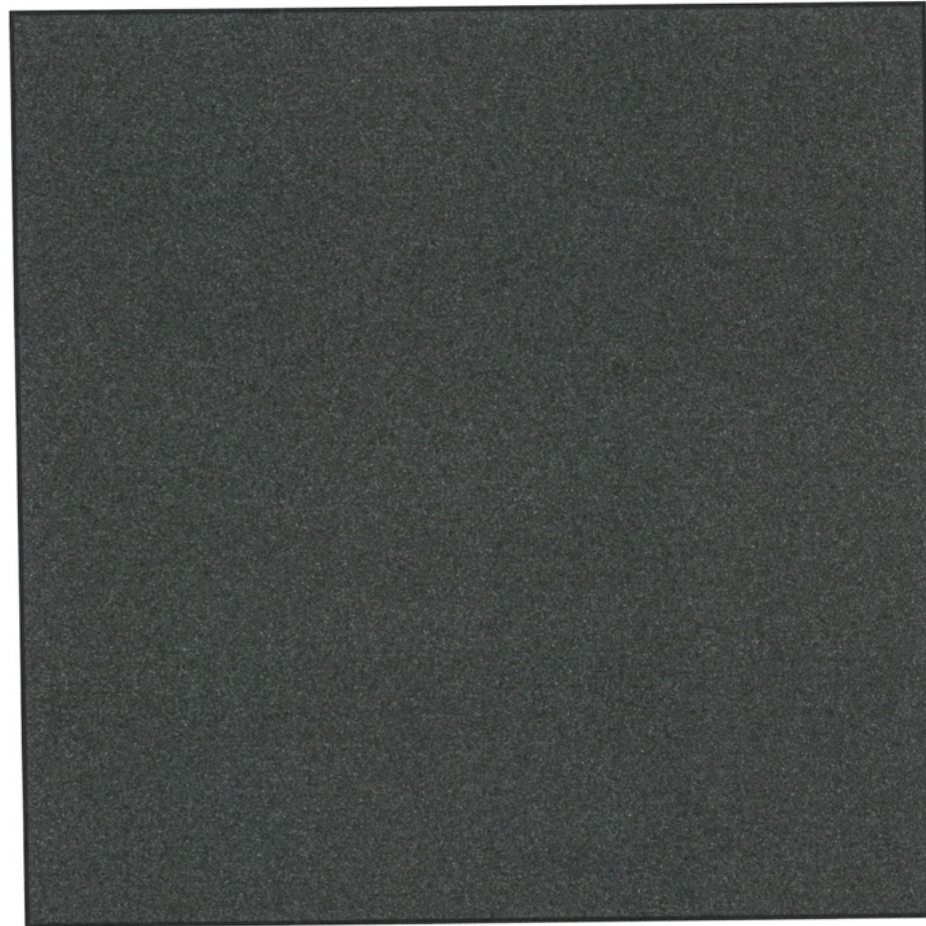
Blurred + just a bit more noise

$S/N = 10^6$ (still way unrealistic)



Deconvolved

M51 is gone, there's just noise!



Deconvolution amplifies
noise! Let's see why.

Noise amplification

- Combination of noise and PSF:

$$\begin{aligned} c(x_1, x_2) &= s(x_1, x_2) + n(x_1, x_2) \\ &= u(x_1, x_2) * r(x_1, x_2) + n(x_1, x_2) \end{aligned}$$

original
(pristine)
image

so: we want to retrieve u from c

Noise amplification

- Combination of noise and PSF:

$$\begin{aligned}c(x_1, x_2) &= s(x_1, x_2) + n(x_1, x_2) \\ &= u(x_1, x_2) * r(x_1, x_2) + n(x_1, x_2)\end{aligned}$$

$$\text{FT} \Rightarrow C(k_1, k_2) = U(k_1, k_2)R(k_1, k_2) + N(k_1, k_2)$$

Now we deconvolve —

Noise amplification

- Combination of noise and PSF:

$$\begin{aligned}
 c(x_1, x_2) &= s(x_1, x_2) + n(x_1, x_2) \\
 &= u(x_1, x_2) * r(x_1, x_2) + n(x_1, x_2) \\
 \Rightarrow C(k_1, k_2) &= U(k_1, k_2)R(k_1, k_2) + N(k_1, k_2) \\
 \Rightarrow U(k_1, k_2) &= \frac{C(k_1, k_2) - N(k_1, k_2)}{R(k_1, k_2)}
 \end{aligned}$$

both decrease with h
(strongly)

$$\Rightarrow \frac{C(k_1, k_2)}{R(k_1, k_2)} - \frac{N(k_1, k_2)}{R(k_1, k_2)}$$

← roughly constant with h !

→ noise is greatly amplified at high h !

(white) noise doesn't have structure

- Solutions?

(1) Throw out high- h modes? Also removes detail from image - -

(2) Downweight (= "filter out") high- h (noise-dominated) modes

Optimal filter

- Find the Fourier filter Φ :

$$\tilde{U}(\mathbf{k}) = \frac{\Phi(\mathbf{k})C(\mathbf{k})}{R(\mathbf{k})}$$

we want to find the Φ that yields the "best" image u

Example: derivation of the optimal filter in the least-squares sense:

- Least-squares:

$$\begin{aligned} \int_{-\infty}^{\infty} |\tilde{u}(x) - u(x)|^2 dx &= \int_{-\infty}^{\infty} |\tilde{U}(k) - U(k)|^2 dk \quad (\text{Parseval's theorem}) \\ &= \int_{-\infty}^{\infty} \left| \frac{\Phi(k)[S(k) + N(k)]}{R(k)} - \frac{S(k)}{R(k)} \right|^2 dk \\ &= \int_{-\infty}^{\infty} |R(k)|^{-2} \left[|\Phi(k) - 1|^2 |S(k)|^2 + |\Phi(k)|^2 |N(k)|^2 \right] dk \end{aligned}$$

we can assume that

$|S(\omega)||N(\omega)| \approx 0$ (that is, signal and noise are uncorrelated with each other)

Wiener filter

- Solution:

$$\frac{d(\dots)}{d\Phi} = 0 \Rightarrow \Phi(k) = \frac{|S(k)|^2}{|S(k)|^2 + |N(k)|^2}$$

optimal filter in a LSQ sense
typically constant in k

(if you don't know your noise level, minimize with this as a constant parameter)

- Alternative:

Starting from

$$\tilde{U}(k) = W(k)C(k)$$

instead of $\tilde{U}(k) = \frac{\Phi(k)C(k)}{R(k)}$

so W(k) is filter and deconvolution in one

$$W(k) = \frac{R(k)^*}{|R(k)|^2 + K(k)}$$

$= \frac{|N(k)|^2}{|U(k)|^2}$ *(try to e.g. fit for a constant parameter K and optimize)*

Wiener filtered

Some detail was lost, but much better than smoothed!



Many other applications

- Fourier transforms can be used to:

- Solve complex integrals
- Find most important modes in a signal (e.g. exoplanetary resonances / flux dips)
- Solve gravitational acceleration of a large number (N) of particles in a periodic volume in $O(N \log N)$ rather than $O(N^2)$
- Many more!

Next week:
Solving differential equations